

CSE498 Project 3: Language-Guided TurtleBot3 Navigation with LocateAnything

Ling Shen

October 4, 2026

Validation outcome: LocateAnything, person, and trash-can runs passed;
chair navigation passed only after operator-assisted observation.

1 System and command flow

This project completes the course detector and adds command-triggered visual grounding to the provided TurtleBot3 navigation framework. Gazebo simulates the room, robot, camera, and laser scanner. ROS 2 nodes process those sensor messages, build a map, interpret a command, and coordinate Nav2 navigation. Gazebo itself does not interpret the language command.

A user publishes a string to `/user_command`, for example *Move to the red chair*. The coordinator pauses exploration, cancels conflicting navigation, waits for Nav2 to become idle, and forwards the command. A launch parameter selects exactly one target-selection mode. Both modes reuse the same target message, approach-pose adapter, coordinator, and Nav2 backend.

User command → coordinator: pause and select mode	
Baseline	LocateAnything
Category/index parser + semantic landmarks	Full description + stationary RGB snapshot
YOLO detections feed object memory	Model box + matched LiDAR + saved TF
Selected instance in <code>map</code> → approach pose → Nav2	
SLAM occupancy map + robot pose support planning and motion	

Baseline selection. The deterministic parser recognizes configured categories, aliases such as “trash can,” and an optional index such as `person_2`. Without an index, it selects the nearest matching landmark relative to the robot’s current map position. An index selects that exact observed memory ID. IDs reflect observation order, not personal identity. The parser does not model color: both red-chair and blue-chair descriptions reduce to category `chair`. This source-level fact does not predetermine the result of a live baseline run.

Visual detection. The completed loading method loads the course’s tracked `tb3det_yolo26n.pt` once; `infer()` returns label, confidence, and pixel bounding box in the wrapper’s existing format. The current configured classes are `person`, `trash_can`, and `chair`, with the supplied 0.40 threshold preserved. The ROS wrapper continues publishing its existing detection and debug-image topics. Before editing, a direct test on a blank 480×640 image emitted the stub warning, with `is_loaded=False` and no detections; the full stub stack was not launched.

Reuse and provenance. Course framework: [Yiyuchen Hu’s course framework](#), commit `a048d1e50e18`. Reused research detector, parser, and geometry: [Ling Shen’s research repository](#), commit `694fff69d878`. Existing model worker dependency: [NVIDIA Eagle](#), commit `8442db3b79f7`; model: [LocateAnything-3B](#). Full commits are in `results.json`. Course history, LICENSE, and NOTICE are preserved. Chair geometry is by Wan Yi Seow/Open Robotics under CC BY 4.0; NOTICE identifies the material changes.

2 From image pixels to a navigation goal

Two different maps. SLAM Toolbox uses laser scans and robot motion to estimate pose and construct an occupancy grid: cells indicate free, occupied, or unknown space. Semantic memory adds labeled object landmarks such as `person_2`. It associates repeated detections and retains map positions. Occupancy mapping answers “where can the robot move?”; semantic memory answers “which observed object is this?” A label alone is neither an obstacle map nor a safe navigation goal.

LocateAnything insertion. In LocateAnything mode, the baseline query node is not launched. A persistent worker in a separate Python environment loads the existing local LocateAnything-3B model. The complete referring expression is preserved: only the leading movement phrase is removed. For the red command the actual prompt was:

Locate a single instance that matches the following description: the red chair.

The model’s selected box, rather than a later category lookup, determines the specific target. Continuous YOLO detections may continue updating ordinary semantic memory, but cannot replace this selected instance. Each selection has its own `locateanything_N` ID.

Coherent geometry. Grounding waits for a stationary robot and captures an actual camera image, `CameraInfo`, a nearby laser scan, odometry, and timestamped transforms. The sensor matching tolerance is 0.15 s; stale images and motion during inference are rejected. Transforms are captured before the slower inference finishes. An old observation is never restamped with the inference-completion time.

Model coordinates use a 0–1000 scale. They are converted to the original $W \times H$ camera pixels by $u = xW/1000$, $v = yH/1000$. For box center (u_c, v_c) and camera intrinsics, the optical ray is

$$\mathbf{r}_c = ((u_c - c_x)/f_x, (v_c - c_y)/f_y, 1).$$

The camera-to-scan rotation gives a planar bearing θ . Nearby valid laser returns inside the box’s angular width provide a median range d . The scan-frame point $(d \cos \theta, d \sin \theta, 0)$ is transformed into `map` using the saved scan-time TF. This reuses the research localizer’s planar approximation; it assumes the requested object intersects the laser plane and ignores the small camera-to-laser translation when forming the ray.

Navigation. The selected metric point is sent directly in `SemanticQueryResult`. The adapter computes a 0.5 m standoff along the current robot-to-target direction in the same frame. Nav2 uses the occupancy map, costmaps, and localization to plan and execute the approach. The coordinator tracks success/failure and automatically re-enables exploration after a delay. In the mapped chair demonstration, exploration nodes are intentionally disabled; the observed “exploration resumed” status is a coordinator state transition, not an exploration-motion test.

Failure behavior. Empty or malformed output, multiple boxes, stale/mismatched sensors, robot motion, unusable laser returns, missing TF, and inference errors produce a clear failure and no new target goal. There is no fallback to another chair. The message’s `confidence=1.0` is a documented transport placeholder, not calibrated model confidence.

3 Two measured LocateAnything demonstrations

A compact room contains two copies of the same course VisitorChair mesh with different red and blue materials. Both are visible from the starting pose and have the original solid collision geometry. This provides a same-class distractor without requiring unseen-object search. The robot is reset between commands, and the map is available before grounding.

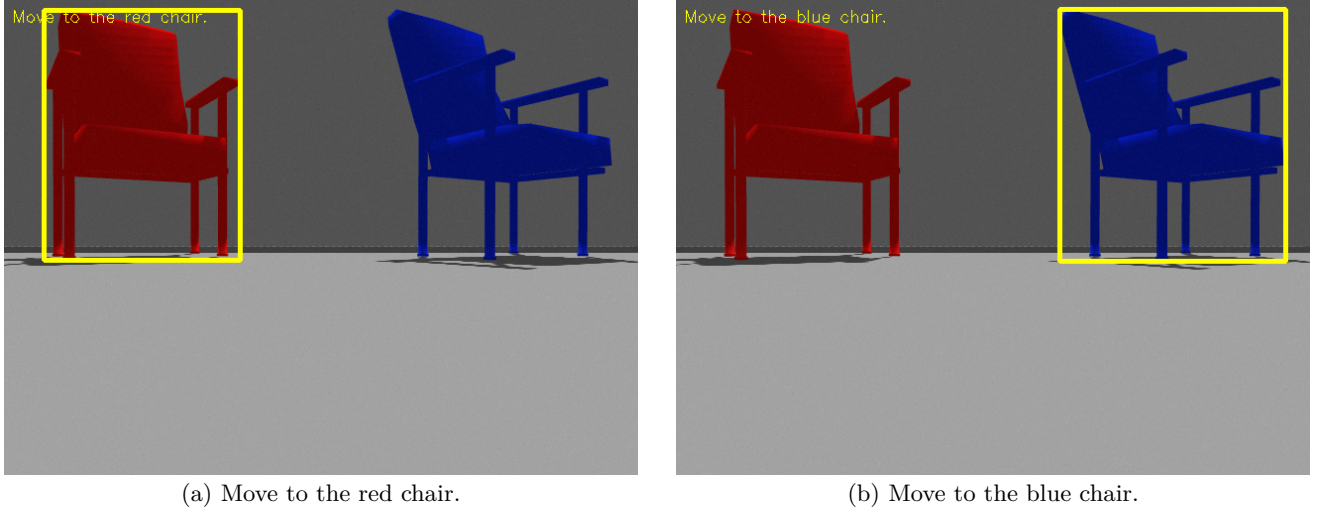


Figure 1: Actual Gazebo camera frames and LocateAnything boxes saved during target selection. The same-class distractor remains visible in each image. These are runtime screenshots, not generated illustrations.

Requested target	Inference (s)	To terminal (s)	Final distance (m)	Arrival ≤ 1.2 m
Red chair	0.168	8.079	1.080	Pass
Blue chair	0.184	7.405	1.170	Pass

Table 1: Latest pair after the cancellation handoff fix, with continuous baseline detection/memory active. Time to terminal includes grounding and navigation. Inference time excludes the 4.697 s model load.

Selection and localization evidence. Both commands reached the model with their color description, produced one box around the requested chair, and preserved that instance through navigation. The selected map coordinates were (0.858, 0.727) for red and (0.776, -0.691) for blue. Source image-to-scan offsets were 94 ms and 10 ms. Selected-point discrepancies from the Gazebo chair origins were 0.339 m and 0.434 m. These include the difference between the observed near surface and the model origin; they are not direct estimates of sensor accuracy.

Arrival measurement. A TARGET_REACHED message alone does not establish course acceptance. After the terminal transition and before resumed motion, the evaluator queries the actual robot and object world poses. The final planar distance to the intended object is compared with the course’s unchanged 1.2 m criterion. The distance to the other chair was 1.623 m for the red run and 1.648 m for the blue run. Gazebo object poses are used only by this evaluator, never by grounding, target selection, or goal generation.

Interpretation. Four earlier attempts also passed; all six records are retained. The latest pair confirms that concurrent baseline detections did not overwrite the grounded instance after the final integration fix. These repeated implementation checks cover two example commands, not a success-rate benchmark. The latest blue result has only 0.030 m margin under the threshold.

4 Validation status, contributions, and limits

Completed checks. All nine packages build; 40 focused tests and an isolated ROS action cancellation regression pass. The live detector self-test passed. Generic `go to person` reached the center person at 0.702 m. Commands for observed IDs `person_0` and `person_1` reached northwest and southeast people at 0.672 m and 0.688 m. The corrected trash-can run passed at 0.726 m. Exploration resumed after each successful baseline run.

Chair limitation. Unaided exploration for 600 s left the chair mislabeled as a trash can, and `go to chair` failed with no chair landmark. After an ordinary memory-based reposition command, an operator turned the camera toward the chair and held still for 22 s. The unchanged detector/memory then relabeled it; a new chair command reached the correct chair at 0.973 m and exploration resumed. This assisted navigation pass does *not* establish the unaided chair acceptance criterion. No GT goal coordinates, injected landmarks, or threshold changes were used.

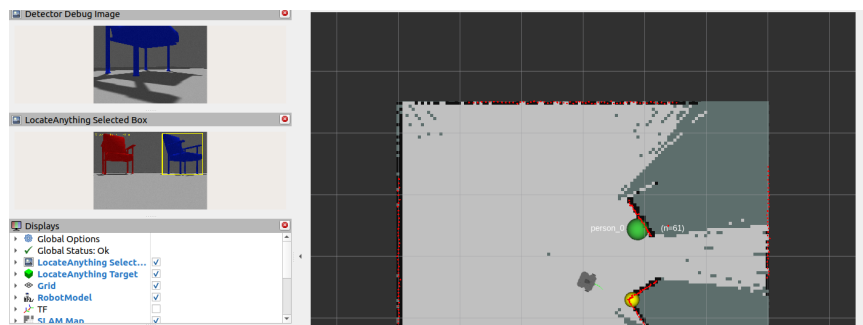


Figure 2: Cropped actual RViz after the second blue-chair run: the saved blue box, yellow selected target, and occupancy map. The separate green `person_0` is the baseline’s false red-chair landmark; it did not replace the `LocateAnything` target.

Retained failure evidence. The live baseline color comparison failed for both red and blue commands with “no active chair in memory”. The recolored red chair was detected as a person, yielding a false person landmark and no selectable chair. Thus the measured contrast includes detector/memory errors on altered appearances; it is not a controlled test of language parsing alone. Separately, the source parser discards color. The blue evaluator also logged an unavailable map TF, preserved in the record.

The first generic-person attempt had missing Gazebo pose queries; its failed evaluator verdict and supplemental 0.751 m measurement are both retained. The first trash-can command selected a mislabeled chair and stopped 2.096 m from the intended trash can. A retry selected the right region but aborted (2.090 m). Waiting for cancellation acknowledgement and Nav2 idle fixed that handoff race; the later trash-can run passed. All attempts, setup notes, and observer errors remain in `results.json`.

My additions. I completed the detector’s two required methods; added the persistent model worker, original-image box parser, stationary synchronized capture, and exact-instance map bridge; added mutually exclusive baseline/LocateAnything modes and two chair material variants; and made bounded backend corrections for robot-relative map selection/standoff, command handling, cancellation, and asynchronous result ordering. The supplied ROS topics and baseline detector return contract are retained. No model training, new planner, or full-room language search was added.

Scope and reproduction. Validation uses Ubuntu 22.04, ROS 2 Humble, an RTX 5090, PyTorch 2.7.1+cu128, Ultralytics 8.4.60, and NumPy 1.26.4. The extension is limited to static, mapped scenes with visible targets that the laser can localize. It does not establish reliable absent-target rejection or arbitrary attribute/relation grounding. README contains the actual launch, reset, and display commands. The final 2–4 minute video is to be recorded manually by the student; no video has been created or linked here.